CIS5200: Machine Learning Spring 2026

26: Post-Training Reasoning Models

Date: April 28, 2026

Acknowledgements. These notes continue the previous lecture on reinforcement learning for language models. They draw on REINFORCE [1], PPO [2], InstructGPT/RLHF [3], DPO [4], GRPO and DeepSeekMath [5], DeepSeek-R1 [6], verifier-based mathematical reasoning [7], process supervision [8], reward-model overoptimization [9], test-time compute scaling [10], chain-of-thought monitoring [11], and prover-verifier games [12]. The goal of these notes is to provide a mathematical intuition behind modern post-training methods.

1 From RL for LLMs to Reasoning Post-training

The previous lecture reduced RL for language models to a simple one-step problem. A prompt is denoted by X, a response by Y, and the language model is the policy

$$\pi_\theta(Y \mid X).$$

A scalar reward $r(X, Y)$ scores the response. For a fixed prompt, the objective is

$$J(\theta) = \mathbb{E}_{Y \sim \pi_\theta(\cdot|X)}[r(X, Y)]. \tag{1}$$

The log-derivative trick gives us

$$\nabla_\theta J(\theta) = \mathbb{E}_{Y \sim \pi_\theta}\left[r(X, Y) \nabla_\theta \log \pi_\theta(Y \mid X)\right]. \tag{2}$$

So a high-reward response receives an MLE-like update, and a low-reward response receives a weak or negative update depending on the estimator that is used.

Reasoning models keep this mechanism, but the item that is scored often falls within a *trace*:

$$Z = (S_1, S_2, \ldots, S_T, Y),$$

where S_t is an intermediate step and Y is the final answer. The trace may contain algebra, code edits, checks of previous work, or alternative attempts. Mathematically, it is still just a sequence sampled from the model:

$$Z \sim \pi_\theta(\cdot \mid X).$$

We introduce Z only to make the internal structure clear.

There are two basic kinds of evaluators. A *verifiable reward* is computed by an external rule. Examples include code passing tests or the answer matching a known solution. A *learned evaluator* is a model

$$\widehat{r}_\phi(X, Z)$$

that is trained to approximate human judgment, correctness, helpfulness, or the validity of a step. Both produce numbers, and both can be used in the same policy-gradient updating procedure. But learned evaluators introduce a new statistical issue where the evaluator has prediction error, and optimization may search for that error, creating reward hacking.

2 Advantages, Critics, and PPO

The estimator in (2) is unbiased, but it is noisy. The same reward scale may mean different things for different prompts. A score of 0.7 may be excellent for a hard problem and poor for an easy problem. What matters for learning is whether a trace was better than the model's typical trace for that prompt.

Let $b(X)$ be any function of the prompt alone. Since

$$\mathbb{E}_{Z\sim\pi_\theta}\left[b(X)\nabla_\theta \log \pi_\theta(Z \mid X)\right] = b(X)\nabla_\theta \sum_Z \pi_\theta(Z \mid X) = 0,$$

we may subtract $b(X)$ without changing the expected gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{Z\sim\pi_\theta}\left[(r(X,Z) - b(X))\nabla_\theta \log \pi_\theta(Z \mid X)\right]. \tag{3}$$

The term

$$A(X,Z) = r(X,Z) - b(X)$$

is called an advantage. It asks whether this trace was better or worse than a baseline for the same prompt. Positive advantage increases the trace probability. Negative advantage decreases it.

The natural baseline is the value function

$$V^\pi(X) = \mathbb{E}_{Z\sim\pi(\cdot|X)}[r(X,Z)].$$

In actor-critic methods, the language model π_θ is the actor, and a separate model V_ψ is the critic. The critic is trained by regression:

$$\mathcal{L}_{\text{critic}}(\psi) = \mathbb{E}\left[(V_\psi(X) - R)^2\right], \tag{4}$$

where $R = r(X,Z)$ is the observed reward. Then

$$\widehat{A} = R - V_\psi(X)$$

estimates whether the sampled trace outperformed the critic's prediction. This is the first modification after REINFORCE– learn a baseline so that the policy update uses relative success, not raw reward.

The second modification is needed because we usually collect traces from an old policy and then perform several gradient steps. Let $Z_i \sim \pi_{\text{old}}(\cdot \mid X_i)$ and define the likelihood ratio

$$\rho_i(\theta) = \frac{\pi_\theta(Z_i \mid X_i)}{\pi_{\text{old}}(Z_i \mid X_i)}. \tag{5}$$

For any function g,

$$\mathbb{E}_{Z\sim\pi_\theta}[g(Z)] = \mathbb{E}_{Z\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(Z \mid X)}{\pi_{\text{old}}(Z \mid X)} g(Z)\right].$$

Thus $\rho_i(\theta)\widehat{A}_i$ is the importance-weighted objective suggested by the policy gradient.

But if ρ_i becomes very large, one sampled trace can dominate the update. PPO controls this by clipping the likelihood ratio:

$$L_{\text{PPO}}(\theta) = \frac{1}{B}\sum_{i=1}^{B} \min\left\{\rho_i(\theta)\widehat{A}_i, \text{clip}(\rho_i(\theta), 1-\varepsilon, 1+\varepsilon)\widehat{A}_i\right\}. \tag{6}$$

The minimum is easiest to understand by cases. If $\widehat{A}_i > 0$, we want to increase the probability of Z_i, but once $\rho_i > 1 + \varepsilon$, the clipped term stops giving extra reward. If $\widehat{A}_i < 0$, we want to decrease the probability of Z_i, but once $\rho_i < 1 - \varepsilon$, the clipped term stops rewarding further movement. PPO is still policy gradient, but it simply prevents a single batch from moving the policy too far.

For language models, there is usually also a reference model π_{ref}, often the SFT model. A common KL-regularized objective has the form

$$L(\theta) = L_{\text{PPO}}(\theta) - \beta \widehat{D_{\text{KL}}}(\pi_\theta \| \pi_{\text{ref}}). \tag{7}$$

The KL term discourages the model from drifting too far from the reference distribution. In the previous lecture, we saw that this appeared as a stability and alignment term. Later we will see another interpretation where KL regularization keeps the policy closer to regions where the evaluator is less likely to be extrapolating.

The next question is what happens if the critic is too expensive to train.

3 GRPO: Relative Rewards without a Critic

For a long reasoning model, training a separate critic can be costly. The critic must read prompts and long traces, predict values, and remain synchronized with a changing policy. GRPO removes this critic by using a simpler baseline– it compares several samples from the same prompt to each other.

Fix a prompt X. Sample a group

$$Z_1, \ldots, Z_G \sim \pi_{\text{old}}(\cdot \mid X)$$

and score each trace:

$$R_i = r(X, Z_i).$$

Define the group mean and standard deviation

$$\bar{R} = \frac{1}{G} \sum_{j=1}^{G} R_j, \qquad s_R = \left(\frac{1}{G} \sum_{j=1}^{G} (R_j - \bar{R})^2 \right)^{1/2}. \tag{8}$$

GRPO uses the normalized group advantage

$$\widehat{A}_i = \frac{R_i - \bar{R}}{s_R + \epsilon}. \tag{9}$$

Then it applies a PPO-style update:

$$L_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^{G} \min \left\{ \rho_i(\theta) \widehat{A}_i, \text{clip}(\rho_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \widehat{A}_i \right\} - \beta \widehat{D_{\text{KL}}}(\pi_\theta \| \pi_{\text{ref}}). \tag{10}$$

The only difference from PPO is the origin of $\widehat{A}_i$. PPO estimates the baseline with a critic whereas GRPO estimates it from the group itself.

Why is this sensible? Suppose the reward is binary. $R_i = 1$ if the answer is correct and $R_i = 0$ otherwise. If exactly k out of the G traces are correct, then before normalization

$$R_i - \bar{R} = \begin{cases} 1 - k/G, & Z_i \text{ correct}, \\ -k/G, & Z_i \text{ incorrect}. \end{cases}$$

So correct traces get positive advantage and incorrect traces get negative advantage. If all traces are wrong or all traces are right, there is little learning signal. The strongest signal appears when the group contains both successes and failures. In that case the model can learn which reasoning choices separated the successful attempts from the unsuccessful ones.

This is why GRPO fits very well within verifiable reasoning tasks. For many math and code problems, one can cheaply decide whether a final answer passes a rule, even if one cannot write the ideal reasoning trace. The model samples several candidate traces, receives simple scores, and updates toward traces that beat the other attempts on the same prompt. In this way, a sparse rule-based reward becomes a training signal for long reasoning.

The statistical role of the group is also powerful. If rewards vary greatly across prompts, a global score threshold is misleading. The group baseline makes the comparison local to the prompt:

$$\text{good} = \text{better than this model's other attempts on this problem.}$$

This is a small mathematical change, but it is central to modern reasoning post-training. It converts absolute rewards into relative evidence about which sampled trace was better.

4 DPO: Preference Optimization as Logistic Regression

Sometimes the training signal is a comparison. For a prompt X, we observe a preferred trace Z^w and a dispreferred trace Z^ℓ:

$$(X, Z^w, Z^\ell),$$
$$Z^w \succ Z^\ell.$$

One route is to train a reward model from such comparisons and then run PPO or GRPO. DPO shows that, under the usual KL-regularized RL objective, we can train the policy directly from the comparisons.

Start with the KL-regularized objective for a fixed prompt:

$$\max_{\pi} \left\{ \mathbb{E}_{Z \sim \pi(\cdot|X)}[r(X, Z)] - \beta D_{\mathrm{KL}}(\pi(\cdot \mid X) \| \pi_{\mathrm{ref}}(\cdot \mid X)) \right\}. \tag{11}$$

Write out the finite-dimensional optimization problem:

$$\max_{\pi} \sum_{Z} \pi(Z \mid X) r(X, Z) - \beta \sum_{Z} \pi(Z \mid X) \log \frac{\pi(Z \mid X)}{\pi_{\mathrm{ref}}(Z \mid X)}$$

subject to $\sum_Z \pi(Z \mid X) = 1$. The first-order condition gives

$$r(X, Z) - \beta \left(\log \frac{\pi^*(Z \mid X)}{\pi_{\mathrm{ref}}(Z \mid X)} + 1 \right) + \lambda = 0.$$

Hence

$$\pi^*(Z \mid X) = \frac{1}{C(X)} \pi_{\text{ref}}(Z \mid X) \exp\left(\frac{1}{\beta} r(X, Z)\right), \tag{12}$$

where $C(X)$ normalizes the distribution. Rearranging,

$$r(X, Z) = \beta \log \frac{\pi^*(Z \mid X)}{\pi_{\text{ref}}(Z \mid X)} + \beta \log C(X). \tag{13}$$

The reward is, up to an X-dependent constant, a log probability ratio between the optimal policy and the reference policy.

Now model preferences by the Bradley-Terry/logistic model:

$$\mathbb{P}(Z^w \succ Z^\ell \mid X) = \sigma\left(r(X, Z^w) - r(X, Z^\ell)\right), \tag{14}$$

where $\sigma(t) = \frac{1}{1+e^{-t}}$. Substituting (13), the normalizing constants cancel:

$$r(X, Z^w) - r(X, Z^\ell) = \beta \left[\log \frac{\pi^*(Z^w \mid X)}{\pi_{\text{ref}}(Z^w \mid X)} - \log \frac{\pi^*(Z^\ell \mid X)}{\pi_{\text{ref}}(Z^\ell \mid X)}\right].$$

DPO replaces the unknown π^* by the trainable policy π_θ and maximizes the likelihood of the observed preferences. The loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(X, Z^w, Z^\ell)} \log \sigma\left(\beta \left[\log \frac{\pi_\theta(Z^w \mid X)}{\pi_{\text{ref}}(Z^w \mid X)} - \log \frac{\pi_\theta(Z^\ell \mid X)}{\pi_{\text{ref}}(Z^\ell \mid X)}\right]\right). \tag{15}$$

This is logistic regression on preference pairs. The score assigned to a trace is

$$s_\theta(X, Z) = \log \frac{\pi_\theta(Z \mid X)}{\pi_{\text{ref}}(Z \mid X)}.$$

The preferred trace should have larger score than the dispreferred trace. The reference correction shows that the model is rewarded for assigning high probability to generic text, and also rewarded for increasing probability relative to the reference model.

DPO and GRPO should now look like two versions of the same idea. GRPO uses online samples and numeric rewards. DPO uses offline winner-loser pairs and a logistic loss.

Up to this point, a trace has been treated as one object. For reasoning, this is too coarse. A long solution can contain a single fatal mistake, or a correct final answer can be reached by luck. The next question is where inside the trace the evaluation should apply.

5 Process Rewards and Credit Assignment

Outcome rewards score the whole trace. If the final answer is correct, the trace receives high reward; if the final answer is wrong, it receives low reward. This is simple, but it creates a credit-assignment problem. The policy-gradient update for a trace is

$$R(X, Z) \nabla_\theta \log \pi_\theta(Z \mid X) = R(X, Z) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(S_t \mid X, S_{<t}), \tag{16}$$

where $R(X, Z)$ is the outcome reward and we suppress final-answer tokenization. Every step receives the same scalar multiplier. A wrong answer caused by step 3 penalizes the whole trace. A lucky correct answer reinforces the whole trace.

Process rewards replace one global score by local scores:

$$R_{\text{proc}}(X, Z) = \sum_{t=1}^{T} r_t(X, S_{\le t}). \tag{17}$$

Then the policy gradient can use return-to-go terms. Define

$$G_t = \sum_{s=t}^{T} r_s(X, S_{\le s}).$$

The update has the form

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{t=1}^{T} G_t \nabla_\theta \log \pi_\theta(S_t \mid X, S_{<t})\right]. \tag{18}$$

A decision at time t is weighted by the rewards that follow it. This is the same policy-gradient mathematics as before, but the evaluator has become more local.

There is also a clean supervised-learning viewpoint. Let us say that:

$$C_t = \{\text{step } t \text{ is valid, assuming previous steps are valid}\}.$$

A process verifier estimates

$$q_\phi(t) = \mathbb{P}_\phi(C_t \mid X, S_1, \ldots, S_t).$$

If step validity is evaluated sequentially, then

$$\mathbb{P}_\phi(C_1 \cap \cdots \cap C_T \mid X, Z) = \prod_{t=1}^{T} q_\phi(t),$$

so

$$\log \mathbb{P}_\phi(C_1 \cap \cdots \cap C_T \mid X, Z) = \sum_{t=1}^{T} \log q_\phi(t). \tag{19}$$

An additive process reward therefore arises as a log-likelihood of stepwise correctness.

If we have step labels $C_{i,t} \in \{0, 1\}$, the process verifier can be trained by cross-entropy:

$$\mathcal{L}_{\text{proc}}(\phi) = -\sum_{i,t} \left[C_{i,t} \log q_\phi(i, t) + (1 - C_{i,t}) \log(1 - q_\phi(i, t))\right]. \tag{20}$$

This is just logistic regression on prefixes of reasoning traces! A classifier trained on labeled examples.

The motivation is important. Outcome supervision asks whether the entire answer good? Process supervision asks smaller questions like "is this step justified?", "did this code edit preserve the intended behavior?", "did this algebraic transformation follow?" Local questions can be easier to label, easier to learn, and more useful for assigning credit.

Process verifiers can be used in two ways. During training, their scores become rewards in PPO or GRPO. During inference, they can score partial or complete traces before the final answer is returned. This leads to verifier-guided search.

6 Verifier-guided Search and Evaluator Error

A model does not have to answer with its first sample. It can generate several candidate traces

$$Z_1, \dots, Z_n \sim \pi_\theta(\cdot \mid X),$$

score them with a verifier v_ϕ, and return

$$Z^* = \arg\max_{1 \le i \le n} v_\phi(X, Z_i). \tag{21}$$

This is the mathematical form of best-of-n generation, re-ranking, verifier-guided reasoning, and many uses of test-time compute.

If the verifier were perfect, the value of search would be immediate. Suppose each sampled trace is correct with probability p, independently, and the verifier selects a correct trace whenever one exists. Then

$$\mathbb{P}(\text{return correct}) = 1 - (1-p)^n. \tag{22}$$

For small p, this is approximately np. Search improves performance because the generator gets more chances and the verifier finds success.

The same calculation also shows the danger. Suppose each candidate is truly correct with probability p. The verifier accepts a correct trace with probability a and accepts an incorrect trace with probability b. Then

$$\mathbb{P}(\text{at least one accepted correct trace}) = 1 - (1-pa)^n, \tag{23}$$

$$\mathbb{P}(\text{at least one accepted incorrect trace}) = 1 - (1-(1-p)b)^n. \tag{24}$$

For small b,

$$1 - (1-(1-p)b)^n \approx n(1-p)b. \tag{25}$$

Thus a small false-positive rate can become important when n is large. Test-time compute amplifies verifier power, but it also amplifies verifier mistakes.

If we condition only on the event that a single sampled trace was accepted, the probability it was correct is

$$\frac{pa}{pa + (1-p)b}. \tag{26}$$

This fraction explains why verifier quality must be judged on the distribution induced by search, not only on random samples. When p is small, even a modest false-positive rate can dominate the accepted set.

The same point can be written as an optimizer's curse. Let $u(X, Z)$ be the true utility of a trace and let the learned evaluator be

$$\widehat{u}_\phi(X, Z) = u(X, Z) + \epsilon(X, Z). \tag{27}$$

Selecting the largest $\widehat{u}_\phi$ means selecting for both large utility and large evaluator error. In the extreme case, suppose all candidates have the same true utility u_0 and

$$\epsilon_i \sim N(0, \sigma^2)$$

independently. Then the selected trace is the one with the largest error. If

$$M_n = \max_{1 \le i \le n} \epsilon_i,$$

then

$$\mathbb{P}(M_n \le t) = \Phi(t/\sigma)^n.$$

The maximum is near the level t for which one sample lies above t:

$$n\mathbb{P}(\epsilon_i > t) \approx 1.$$

Using the Gaussian tail approximation gives

$$\mathbb{E}[M_n] \approx \sigma\sqrt{2\log n}. \tag{28}$$

Even when search cannot improve true quality, it can make the selected trace look better and better to the evaluator.

A slightly richer Gaussian model separates true variation from noise. Let

$$U_i = u(X, Z_i) \sim N(0, \tau^2), \qquad \epsilon_i \sim N(0, \sigma^2), \qquad S_i = U_i + \epsilon_i.$$

The evaluator observes S_i. Since (U_i, S_i) are jointly Gaussian,

$$\mathbb{E}[U_i \mid S_i = s] = \frac{\mathrm{Cov}(U_i, S_i)}{\mathrm{Var}(S_i)} s = \frac{\tau^2}{\tau^2 + \sigma^2} s.$$

If $i^* = \arg\max_i S_i$, then approximately

$$\mathbb{E}[U_{i^*}] \approx \frac{\tau^2}{\sqrt{\tau^2 + \sigma^2}} \sqrt{2\log n}, \tag{29}$$

$$\mathbb{E}[\epsilon_{i^*}] \approx \frac{\sigma^2}{\sqrt{\tau^2 + \sigma^2}} \sqrt{2\log n}. \tag{30}$$

When $\tau^2 \gg \sigma^2$, search mostly finds genuinely better traces. When σ^2 is large, search increasingly finds evaluator mistakes. This is the mathematical core of reward hacking, where optimization pressure turns prediction error into an incentive.

This also gives a new interpretation of the KL term

$$\max_{\pi} \left\{ \mathbb{E}_{Z \sim \pi}[\widehat{u}_\phi(X, Z)] - \beta D_{\mathrm{KL}}(\pi(\cdot \mid X) \| \pi_{\mathrm{ref}}(\cdot \mid X)) \right\}. \tag{31}$$

KL regularization is not only a stabilizer, but it is a region of trust for the evaluator. It discourages the policy from moving too far into regions where the evaluator is extrapolating and its errors may be larger.

7 Checkable Reasoning

The previous section says that learned evaluators must be designed for the regime in which they will be used. A verifier used under search is part of an optimization loop. The final design principle is therefore checkability.

Process supervision is one route. Instead of asking whether a whole proof, program, or derivation is good, ask local questions:

$$\text{Is } S_t \text{ a valid next step given } X, S_{<t}?$$

This changes the statistical problem. A single step may be easier to label, easier to learn, and harder to game than an entire solution. In notation, the evaluator moves from

$$\widehat{u}_\phi(X, Z)$$

to a decomposition

$$\sum_{t=1}^{T} \widehat{r}_{\phi,t}(X, S_{\le t}).$$

The goal here is to make the correctness of the trace checkable.

Chain-of-thought monitoring is another route. If a model exposes a reasoning trace, a monitor can classify the trajectory:

$$m_\psi(X, Z, A) \approx \mathbb{P}(\text{serious error or reward hacking} \mid X, Z, A), \tag{32}$$

where A may include code edits or external actions. This is again supervised learning, where there is a classifier on trajectories. But the same warning applies. If we directly optimize the model to avoid monitor flags, the monitor can become the new learned evaluator. The model may learn to fix the bad behavior, or it may learn to hide the evidence used by the monitor.

A more explicit way to train for checkability is to use hard negatives. Let $v_\phi(X, Z)$ be a verifier. Let Z^+ denote correct, checkable traces and Z^- denote incorrect but plausible traces. Train

$$\mathcal{L}_{\text{check}}(\phi) = \mathbb{E}_{Z^+}[-\log v_\phi(X, Z^+)] + \lambda \mathbb{E}_{Z^-}[-\log(1 - v_\phi(X, Z^-))]. \tag{33}$$

The second term matters because an optimizer will not usually find random mistakes. It will find plausible mistakes that fool the verifier.

This connects to prover-verifier games. A helpful prover tries to produce correct traces accepted by the verifier. A sneaky prover tries to produce incorrect traces accepted by the verifier:

$$\max_{\text{helpful prover}} \quad \mathbb{P}(v_\phi(X, Z) = 1 \text{ and } Z \text{ is correct}), \tag{34}$$

$$\max_{\text{sneaky prover}} \quad \mathbb{P}(v_\phi(X, Z) = 1 \text{ and } Z \text{ is incorrect}). \tag{35}$$

The sneaky prover is useful because it generates hard negatives for the verifier. If

$$\alpha_\phi = \mathbb{P}_{Z^-}(v_\phi(X, Z^-) = 1)$$

is the probability that an incorrect plausible trace is accepted, then over n attempts,

$$\mathbb{P}(\text{some incorrect trace is accepted}) = 1 - (1 - \alpha_\phi)^n \approx n\alpha_\phi. \tag{36}$$

So a verifier for a heavily searched system must have very small false-positive probability on hard negatives, not just good average accuracy.

References

[1] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[3] L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.

[4] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS*, 2023.

[5] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.

[6] D. Guo et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.

[7] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.

[8] H. Lightman et al. Let's Verify Step by Step. arXiv:2305.20050, 2023.

[9] L. Gao, J. Schulman, and J. Hilton. Scaling Laws for Reward Model Overoptimization. arXiv:2210.10760, 2022.

[10] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314, 2024.

[11] B. Baker et al. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025.

[12] J. H. Kirchner et al. Prover-Verifier Games improve legibility of LLM outputs. arXiv:2407.13692, 2024.